Teleoperation: Self-Driving Car Project

Capstone - ECE4773

Lili Nguyen

Van Lang

Ousmane Segba Kone

Pundit Vorakitolan

Team Introductions

Lili Nguyen (PM)

Van Lang (Software)

Ousmane Segba Kone (Software)

Pundit Vorakitolan (Hardware)

OK

MiLab Introduction

MiLa
Mobility Intelligence Lab

Research lab led by Dr. Bin Xu, research assistant Professor at the University of Oklahoma

Focuses on intelligent mobility systems and data-driven vehicle technologies.

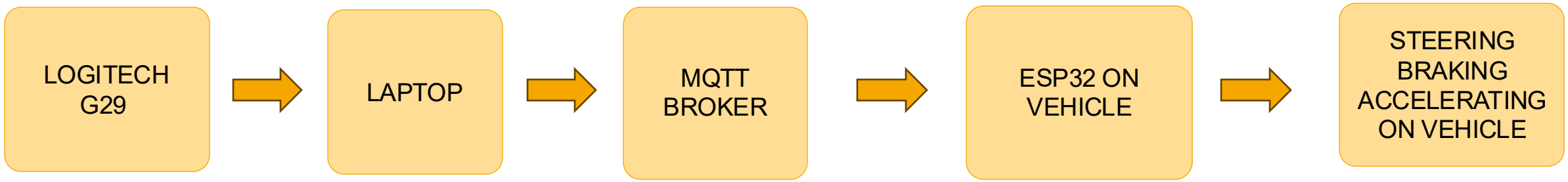
OK

Problem Statement: MiLab lacks a reliable long-distance communication system to remotely control a DBW-enabled car using a driving simulator.

Who it Affects ?	The Pain	What To Do ?
MiLab researchers & students	Unreliable long-distance simulator-to-DBW control.	Develop low-latency simulator-to-vehicle control for remote testing.

OK

System Overview



VL

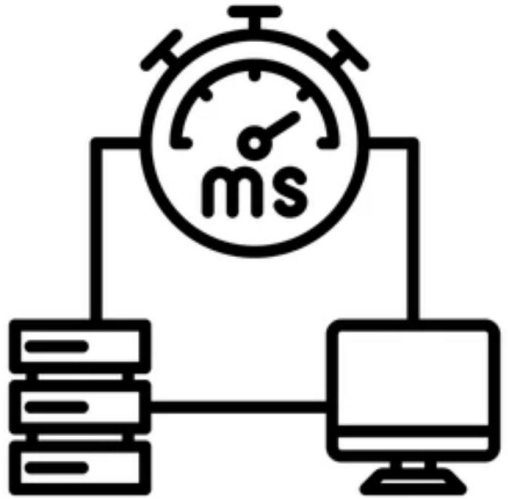
Quantitative Design Requirements

- Input Precision for Bluepad32
  - Requirement: The system must map 16 bit analog inputs from the Logitech G29
  - Metric: 256 discrete steps for the throttle/brake and steering (0 to 255)
- Data Throughput
  - Requirement: Maintain consistent telemetry stream
  - Metric: Update frequency at 100Hz to ensure smooth control loops

VL

Performance Objectives

- Latency
  - Objective: minimize the latency from the driving simulator
  - Target: Latency < 200ms
- Connection Stability
  - Objective: Maintain connection without drops
  - Target: Successful packet rates during the driving session



VL

Source Code

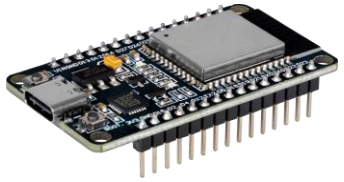
- Bluepad32 uses a virtual gamepad
- Uni_gamepad.h
  - Data structure for steering, throttle, and braking
- Uni_controller.h
  - Manages bluetooth and connection handshakes
- Take these principles from Bluetooth and apply them to our system

Technical Approach

PV

serial

wireless



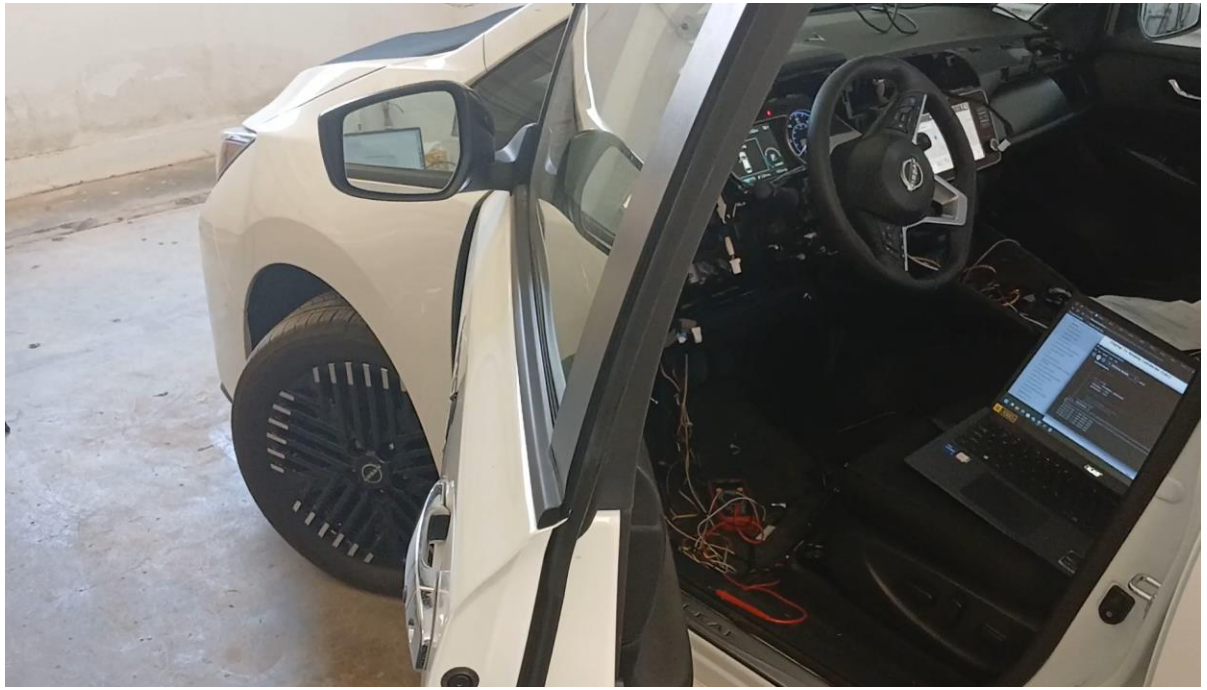
- Get a working MVP, then rapidly iterate on it
  - MVP will likely be similar to the other two capstone teams
    - We all have the same hardware so no Bill of Materials
  - Ultimate goal is to reduce latency; we want to test as many differing ideas as possible
  - Allows for course correction and evolving to help the lab where it needs it the most
- MVP approach
  - Interface between a Logitech G29 and an ESP32 using python
  - Use the lab's incumbent protocol (MQTT) to transmit messages via intermediary (broker)
  - Once messages arrive at the car ESP32/relay/DAC PCB board, hand-off to DBW team
- Improving the MVP
  - Send telemetry/feedback data from the car
  - Timestamp and track down/fix root cause of latency

Focus: use software to wirelessly interface between a driving simulator and a car. Iterate on that MVP, optimize latency, and control the car well enough to drift.

PV

Technical Approach

- We have a DBW (drive-by-wire) system to integrate our system into
  - We also have a PCB with an ESP32/relays/DAC on the car side; we also want to design a smaller board to house those components
- Our main technical problem to solve after building the MVP is the latency between device and car, illustrated by the Bluetooth solution



Our driving sim/ESP32 duo integrates into this system

Latency – it's a problem!

PV

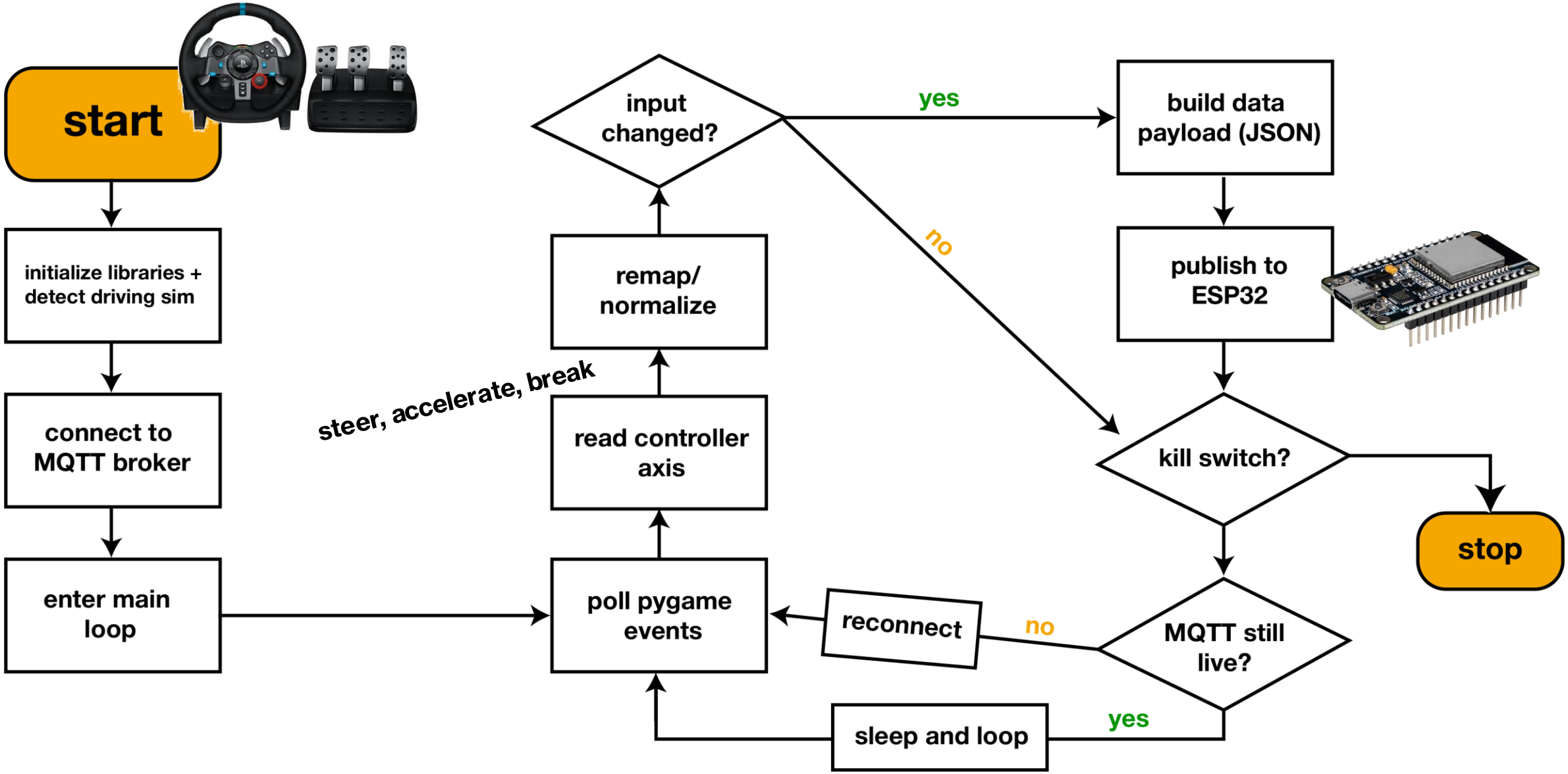
Latency & Upgrades

- A PS5 controller over bluepad32 should only add a few milliseconds
  - Our 1-2 sec latency comes from another source
  - We should look at timestamp logging to identify the issue
  - Adding another network layer will only exacerbate the problem
  - Slow ESP32/car ECU processing likely suspect?
- Start with pygame and upgrade to psydl2 when we need to optimize latency
- Add features like force feedback on the wheel

Latency – it's a problem!

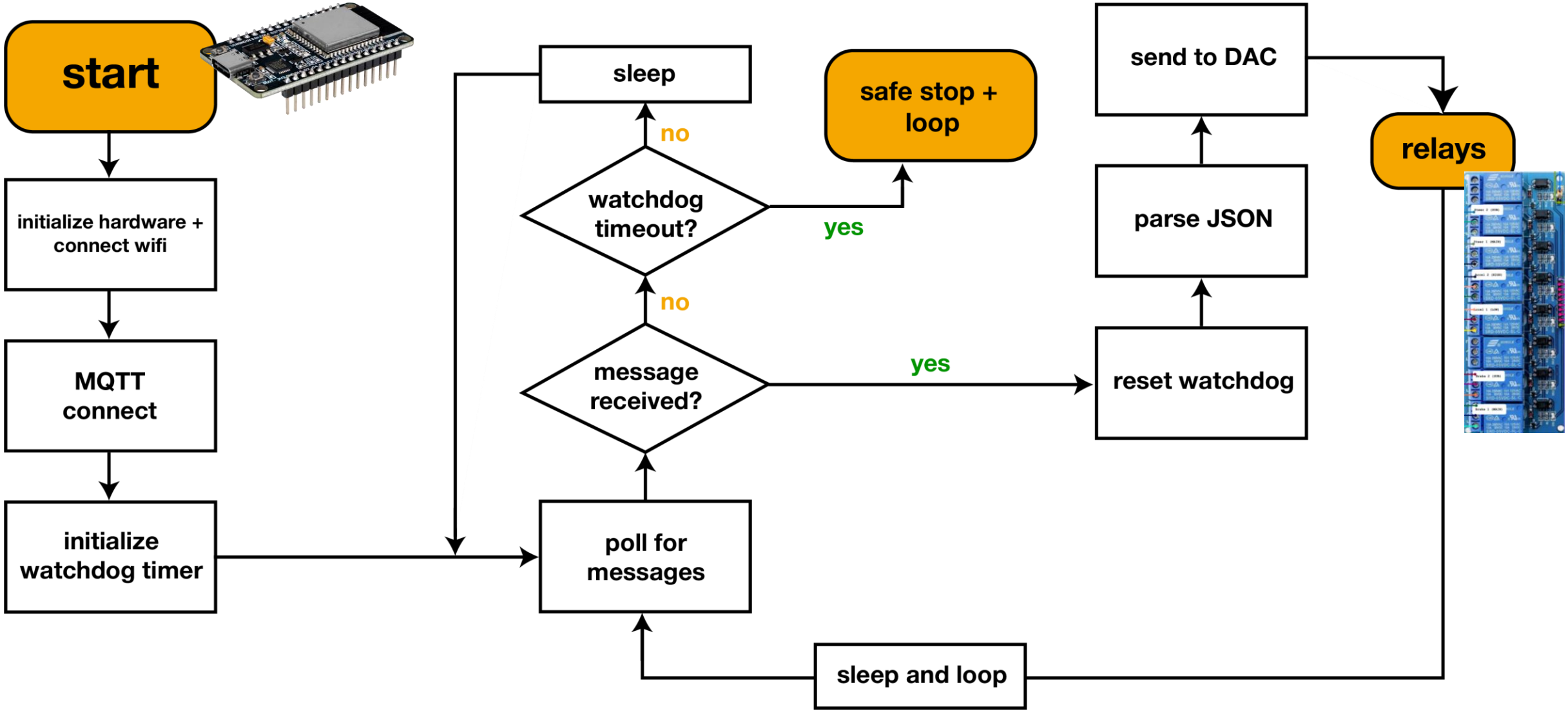
PV

Driving Sim Software Block Diagram



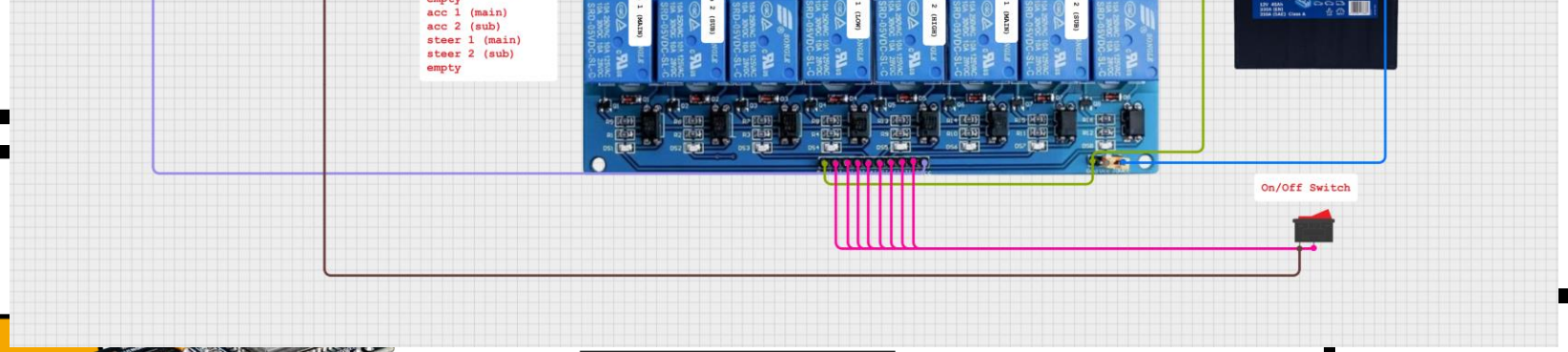
PV

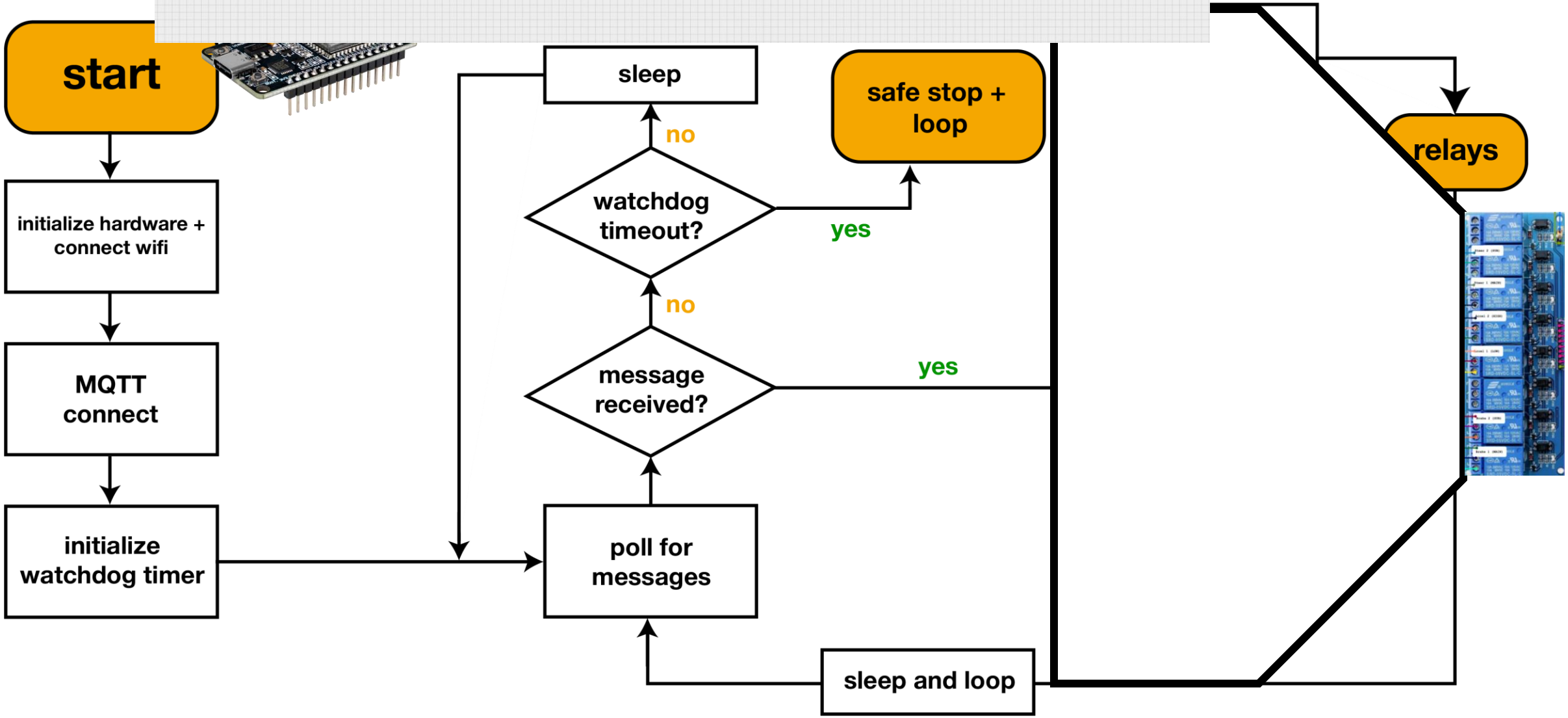
ESP32 Software Block Diagram



ESP

PV

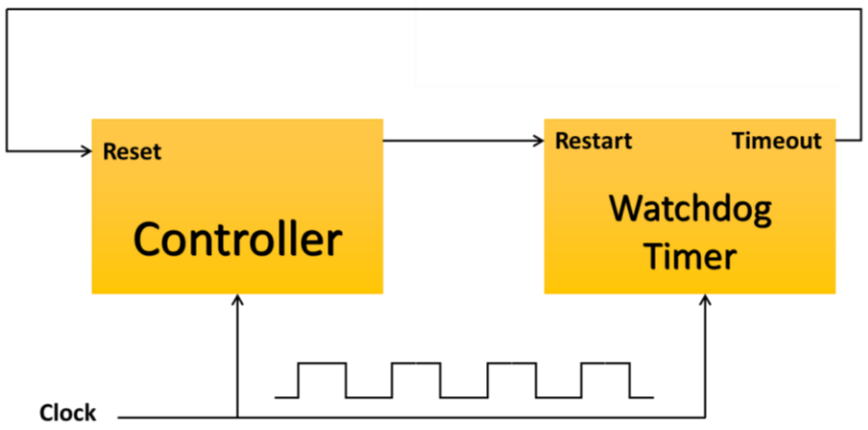

acc 1 (main)
acc 2 (sub)
steer 1 (main)
steer 2 (sub)
empty
On/Off Switch


start
initialize hardware + connect wifi
MQTT connect
initialize watchdog timer
sleep
no
watchdog timeout?
yes
safe stop + loop
no
message received?
yes
poll for messages
sleep and loop
relays

LN

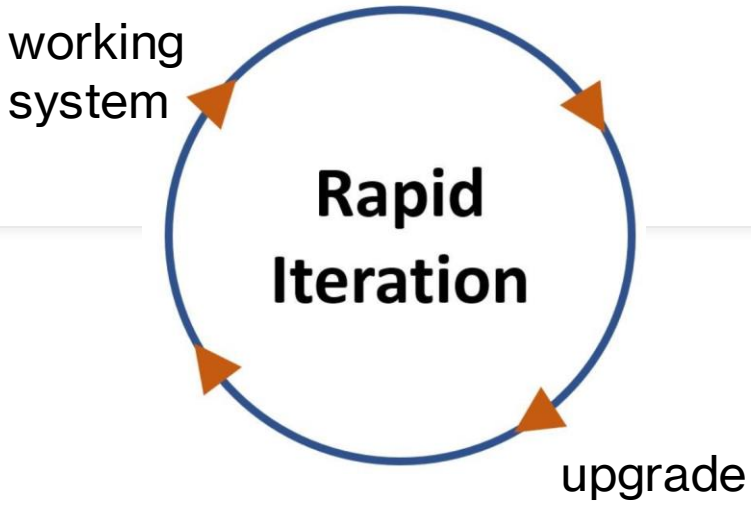
Safety Considerations

- Controlled Testing
- Hard Stop
- Operator Awareness
- Comms Loss Behavior



LN

Major Tradeoff Issues



- Simplicity -> Complexity
- MQTT vs other protocols
  - Other protocols/methods of remotely communication may reduce latency
  - MQTT is actively being used by this lab
  - Better to start simple and add complexity later
- G29 vs PS5 controller
  - The G29 is more reliable and provides a closer simulation to real life
  - Instead of Bluetooth, we can use a more customizable communication protocol

LN

Cost/Budget

- Main costs are small parts:
  - ESP32 boards
  - wires, connectors, adapters
  - power parts (regulator, fuse, switch)
- Cost control
  - Use lab's existing DBW system, share the same expensive hardware with other teams
- Budget risk: extra parts if hardware breaks
- Plan: pre-test early to avoid damage and reduce rework

LN

Milestone Chart/Schedule

1

Preliminary Design Phase (2/08–2/22)

- Define system architecture and requirements
- Draft interfaces, message schema, and initial prototype plan

2

Critical Design Phase (2/22–4/01)

- Implement and integrate core system (end-to-end path)
- Add safety behavior (timeouts, E-stop) and validation tests

3

Final Demonstration Phase (4/01–4/22)

- Plan to run stable demo, measure latency
- Execute final test checklist and capture results
- **Prepare Project Repository**
- **Design Archives**
- **Written Report**

Q&A